

GUIDE TECHNIQUE & RÉPERTOIRE DES FONCTIONS

Application Android : Bus Fianarantsoa (Madagascar)

Documentation intégrale du code source : rôle de chaque classe, interface, fonction, algorithme d'itinéraire et composant Jetpack Compose.

Architecture & Stack

- Jetpack Compose & Material 3
- Pattern MVVM & Clean Architecture
- Room Database (100% Hors-Ligne)
- Kotlin Coroutines & StateFlow
- Tests Robolectric & Roborazzi

Réseau Urbain Intégré

- 10 Lignes urbaines complètes
- 35+ Arrêts géolocalisés
- Calculateur de correspondances
- Plan vectoriel Canvas interactif
- Simulateur & Lexique Malagasy

Sommaire des Modules Expliqués

1. Architecture & Flux Unidirectionnel (UDF) : Fonctionnement de la réactivité Compose.
2. Source de Données & Moteur (FianarBusData.kt) : Les 10 lignes et l'algorithme d'itinéraire.
3. Persistance Room (BusDao, BusDatabase, BusEntities) : Requêtes SQL et persistance SQLite.
4. Couche Repository (BusRepository.kt) : Source unique de vérité et opérations CRUD.
5. Logique Métier & ViewModel (BusViewModel.kt) : État UI immuable et fonctions de contrôle.
6. Activité Principale (MainActivity.kt) : Scaffold, barre de navigation et mode Edge-to-Edge.
7. Composants Graphiques (ui/components/) : Cartes, modales, plan Canvas, filtres et guide.
8. Thème, Accessibilité & Tests : Palette M3, conformité WCAG, cibles 48dp et tests Robolectric.

1. ARCHITECTURE GLOBALE & FLUX DE DONNÉES

Organisation Clean Architecture & MVVM

L'application 'Bus Fianarantsoa' est construite selon les standards recommandés par Google pour Android moderne. Elle garantit une séparation stricte entre l'interface utilisateur, la logique métier et la persistance des données.

Le Flux Unidirectionnel des Données (UDF)

Dans ce modèle, l'état descend vers les composants graphiques et les événements remontent vers le ViewModel :

- **1. Interaction Utilisateur** : L'utilisateur clique sur une ligne, saisit une recherche ou modifie un filtre dans un composable Compose.
- **2. Appel de Fonction ViewModel** : Le composable appelle directement la méthode correspondante du BusViewModel (ex: viewModel.selectLine(line)).
- **3. Traitement en Coroutine** : Le ViewModel lance une coroutine dans le scope 'viewModelScope', interagit avec le 'BusRepository' et met à jour l'objet d'état immuable 'BusUiState'.
- **4. Émission par StateFlow** : Le 'StateFlow<BusUiState>' notifie automatiquement les observateurs de la nouvelle valeur.
- **5. Recomposition Compose** : L'UI observe l'état via 'collectAsStateWithLifecycle()' et ne redessine que les éléments dont l'état a changé, garantissant une fluidité maximale.

Arborescence des Fichiers Clés du Projet

- data/model/BusModels.kt : Définition des structures de données métiers (BusLine, BusStop, ItineraryPlan...)
- data/datasource/FianarBusData.kt : Données statiques des 10 lignes et algorithmes d'itinéraire
- data/local/ : Gestion SQLite Room (BusEntities.kt, BusDao.kt, BusDatabase.kt)
- data/repository/BusRepository.kt : Source unique de vérité et opérations CRUD
- ui/viewmodel/BusViewModel.kt : Gestionnaire d'état réactif de l'UI
- ui/components/ : Composants visuels modulaires (BusLineCard, Modal, Canvas Map, etc.)
- MainActivity.kt : Activité principale, barre de navigation M3 et gestion plein écran

2. Répertoire Exhaustif de Toutes les Fonctions du Projet

2.1. Source de Données & Moteur Algorithmique (FianarBusData.kt)

Ce singleton regroupe les données du terrain et les calculs de trajet sans faire appel au réseau.

Fonction & Signature	Type de retour	Rôle & Explication concrète
getLines()	List<BusLine>	Retourne la liste immuable des 10 lignes urbaines de Fianarantsoa (L1 à L10) avec leurs arrêts ordonnés, terminus et fréquences.
getStops()	List<BusStop>	Fournit l'annuaire des 35+ arrêts physiques avec leur zone (Haute-Ville, Université...) et repères visuels.
getNetworkNodes()	List<NetworkNode>	Fournit les coordonnées normalisées (X, Y de 0.0 à 1.0) des stations pour le dessin vectoriel sur le Canvas.
getStudentTips()	List<StudentTip>	Fournit les conseils et astuces pour les étudiants desservant le campus d'Andrainjato.
getVocabularyList()	List<VocabularyItem>	Retourne les expressions bilingues Malagasy/Français des transports en taxi-be.
findStopByName(name)	BusStop?	Recherche insensible à la casse d'un arrêt dans le catalogue. Retourne l'objet ou null.
findLinesPassingByStop(stopName)	List<BusLine>	Détermine toutes les lignes qui desservent un arrêt donné (utilisé pour identifier les correspondances).
calculateItinerary(depart ure, arrival)	List<ItineraryPlan>	Moteur d'itinéraire en 2 phases : recherche d'abord les lignes directes (500 Ar), puis explore les carrefours hubs pour proposer des correspondances (1000 Ar).

2.2. Interface d'Accès aux Données SQLite (BusDao.kt)

Définit les contrats d'accès à la base de données Room avec requêtes SQL réactives.

Méthode DAO	Type de retour	Rôle & Requête SQL sous-jacente
<code>getAllFavorites()</code>	Flow<List<FavoriteItem>>	SELECT * FROM favorites ORDER BY createdAt DESC. Émet la liste des favoris dès modification.
<code>getFavoritesByType(type)</code>	Flow<List<FavoriteItem>>	SELECT * FROM favorites WHERE type = :type. Filtre les favoris de type 'LINE' ou 'STOP'.
<code>getFavoriteById(itemId, type)</code>	FavoriteItem? (suspend)	SELECT * FROM favorites WHERE itemId = :itemId AND type = :type LIMIT 1. Vérifie si l'élément est favori.
<code>insertFavorite(favorite)</code>	Unit (suspend)	INSERT OR REPLACE INTO favorites. Sauvegarde un favori en base SQLite.
<code>deleteFavorite(favorite)</code>	Unit (suspend)	DELETE FROM favorites. Supprime l'entité favori de la base SQLite.
<code>deleteFavoriteById(itemId, type)</code>	Unit (suspend)	DELETE FROM favorites WHERE itemId = :itemId AND type = :type. Supprime par clé composite.
<code>getAllNotes()</code>	Flow<List<TripNote>>	SELECT * FROM trip_notes ORDER BY createdAt DESC. Émet les mémos personnels en continu.
<code>insertNote(note)</code>	Unit (suspend)	INSERT OR REPLACE INTO trip_notes. Enregistre un mémo de voyage dans SQLite.
<code>deleteNoteById(id)</code>	Unit (suspend)	DELETE FROM trip_notes WHERE id = :id. Supprime une note par son identifiant unique.

2.3. Gestionnaire de Base de Données (BusDatabase.kt)

Méthode	Type de retour	Rôle & Explication
<code>busDao()</code>	BusDao	Méthode abstraite que Room génère automatiquement pour instancier le DAO.
<code>getInstance(context)</code>	BusDatabase	Singleton thread-safe avec double vérification (@Volatile et synchronized). Crée ou retourne l'instance de 'fianar_bus.db'.

2.4. Couche Repository (BusRepository.kt)

Centralise l'accès aux données mémoire et SQLite, agissant comme source unique de vérité.

Méthode du Repository	Type de retour	Rôle & Explication concrète
getAllLines()	List<BusLine>	Fournit toutes les lignes de bus disponibles dans l'application.
getAllStops()	List<BusStop>	Fournit l'annuaire complet des arrêts de bus de la ville.
searchLines(query, category)	List<BusLine>	Filtre les lignes selon le texte saisi et la catégorie sélectionnée (Campus, Marché, Hôpital, Gare).
getItinerary(departure, arrival)	List<ItineraryPlan>	Délègue le calcul de trajet au moteur algorithmique de FianarBusData.
getAllFavorites()	Flow<List<FavoriteItem>>	Expose le flux réactif de tous les favoris de l'utilisateur.
isFavorite(itemId, type)	Flow<Boolean>	Retourne un flux booléen indiquant si un élément précis est enregistré en favori.
toggleFavorite(itemId, type, title, subtitle)	Unit (suspend)	Bascule automatique : supprime l'élément s'il existe déjà, ou l'insère s'il n'existe pas.
getAllNotes()	Flow<List<TripNote>>	Expose le flux réactif de tous les mémos personnels enregistrés.
addNote(title, lineOrStop, text)	Unit (suspend)	Construit une entité TripNote avec horodatage et l'insère en base SQLite via le DAO.
deleteNote(id)	Unit (suspend)	Supprime une note de la base SQLite à partir de son identifiant.

2.5. Logique Métier & Contrôle d'État (BusViewModel.kt)

Survit aux rotations d'écran et gère l'état 'BusUiState' complet de l'application.

Méthode du ViewModel	Paramètres	Rôle & Impact sur l'Interface
setSelectedTab(tab)	tab: AppTab	Change l'onglet actif affiché (LIGNES, ITINÉRAIRE, PLAN, FAVORIS, GUIDE).
setSearchQuery(query)	query: String	Met à jour la recherche textuelle et actualise la liste des lignes filtrées.
setCategoryFilter(filter)	filter: LineCategoryFilter	Active un filtre thématique (Toutes, Université, Marché, Hôpital, Gare).
getFilteredLines()	Aucun	Calcule et renvoie la liste des lignes respectant les critères de recherche et filtres.
selectLine(line)	line: BusLine?	Ouvre la feuille modale de détail avec la ligne passée en paramètre, ou la ferme si null.
toggleDirection()	Aucun	Inverse le sens d'affichage des arrêts de la ligne sélectionnée (Aller !À Retour).
toggleFavoriteLine(line)	line: BusLine	Ajoute ou retire la ligne des favoris et affiche un message de confirmation Snackbar.
toggleFavoriteStop(stop)	stop: BusStop	Ajoute ou retire l'arrêt spécifié des favoris.
setDepartureStop(stop)	stop: String	Sélectionne l'arrêt de départ et lance le recalcul si l'arrivée est déjà choisie.
setArrivalStop(stop)	stop: String	Sélectionne l'arrêt d'arrivée et lance le recalcul d'itinéraire.
swapItineraryStops()	Aucun	Intervertit le départ et l'arrivée en un seul clic et recalcule immédiatement l'itinéraire inverse.
searchItinerary()	Aucun	Exécute le calcul d'itinéraire complet et met à jour 'itineraryResults' dans l'état UI.
openAddNoteDialog(ref)	ref: String	Ouvre la boîte de dialogue d'ajout de mémo avec pré-remplissage éventuel de la ligne.
closeAddNoteDialog()	Aucun	Ferme la boîte de dialogue d'ajout de mémo et réinitialise les champs de saisie.
setNoteTitle(title)	title: String	Met à jour le titre du mémo en cours de saisie.
setNoteLineOrStop(ref)	ref: String	Met à jour la référence de ligne ou d'arrêt associée au mémo.
setNoteText(text)	text: String	Met à jour le contenu textuel du mémo.
saveNote()	Aucun	Valide les champs, insère la note dans Room via viewModelScope et affiche un Snackbar.
deleteNote(id)	id: Int	Supprime une note existante de la base SQLite.
clearSnackbar()	Aucun	Efface le message Snackbar après affichage pour éviter les apparitions répétées.

2.6. Activité Principale & Composants de Présentation

Composant Compose	Fichier source	Rôle & Spécificités d'implémentation
onCreate	MainActivity.kt	Point d'entrée Android. Active le mode Edge-to-Edge et attache BusFianarApp.
BusFianarApp	MainActivity.kt	Scaffold principal, barre de navigation NavigationBar M3 avec 5 onglets.
HeaderHeroBanner	MainActivity.kt	Bannière d'en-tête avec image urbaine, dégradé contrasté et titre dynamique.
BusLinesView	BusLineCard.kt	Écran annuaire des lignes : barre de recherche, filtres défilants et liste verticale.
BusLineCard	BusLineCard.kt	Carte Material 3 d'une ligne avec badge couleur, fréquence, prix et bouton favori.
BusLineDetailModal	BusLineDetailModal.kt	Feuille modale inférieure : timeline verticale des arrêts et correspondances.
ItineraryFinderView	ItineraryFinderView.kt	Calculateur : ExposedDropDownMenuBox, inversion rapide et cartes de solutions.
NetworkMapView	NetworkMapView.kt	Plan schématique vectoriel Canvas avec détection tactile pointerInput.
FavoritesNotesView	FavoritesNotesView.kt	Écran scindé : favoris enregistrés et carnet de mémos avec FloatingActionButton.
GuideAndFareView	GuideAndFareView.kt	Simulateur de budget mensuel à deux curseurs Slider et lexique Malagasy.

3. Algorithmes de Calcul d'Itinéraire & Rendu Canvas

Fonctionnement de l'Algorithme de Routage

La fonction calculateItinerary résout la recherche de trajet sans nécessiter de serveur distant :

- **Étape 1 : Trajets Directs** : Parcourt les 10 lignes du réseau. Si les deux arrêts figurent sur la même ligne, l'algorithme calcule le nombre d'arrêts intermédiaires ($|indexArr - indexDep|$). Le temps estimé est calculé à raison de 3 minutes par arrêt, et le tarif officiel de 500 Ariary est appliqué.
- **Étape 2 : Trajets avec Correspondance** : Si aucun bus direct ne relie les deux arrêts, l'algorithme explore les pôles d'échange stratégiques de Fianarantsoa (Tsianolondroa, Ampasambazaha, Antarandolo, Beravina). Il identifie une ligne A reliant le départ au pivot, et une ligne B reliant le pivot à l'arrivée. Le coût cumulé est de 1000 Ariary et un temps de battement de 8 minutes est automatiquement ajouté pour la correspondance.

Rendu Vectoriel et Détection Tactile sur le Canvas

Le composant 'NetworkMapView.kt' dessine la carte schématique directement sur le Canvas matériel :

- **Dessin des Lignes (drawCorridor)** : Relie les stations successives par des segments colorés selon la teinte officielle de la ligne.
- **Détection Tactile (pointerInput)** : Intercepte les touches de l'utilisateur. Une formule de distance euclidienne calcule l'écart entre le point touché (x, y) et chaque station. Si la distance est inférieure au seuil de tolérance (35dp), la station est sélectionnée et sa fiche d'information s'affiche dynamiquement en bas de l'écran.

4. Thème Graphique, Accessibilité & Tests Automatisés

Système de Design Material 3 & Accessibilité

- **Palette de Couleurs** : Inspirée des terres ocres et de la végétation de Fianarantsoa : Vert émeraude (#004D40), Ocre orangé (#E65100), Brique (#D32F2F).

- **Cibles Tactiles** : Toutes les zones interactives (boutons, icônes, éléments de liste) mesurent au moins 48dp x 48dp conformément aux directives d'accessibilité Android.
- **Descriptions Sémantiques** : Chaque composant Icon dispose d'un paramètre 'contentDescription' explicite pour les lecteurs d'écran (TalkBack).
- **Modifier.testTag** : Des identifiants uniques sont assignés à tous les composants majeurs (navigation_tab_*, search_input, swap_stops_button...) pour faciliter les tests automatisés.

Tests Automatisés du Projet

- **ExampleRobolectricTest.kt** : Exécute des tests locaux sur la JVM pour vérifier le chargement des ressources Android et des données de transport sans émulateur physique.
 - **GreetingScreenshotTest.kt** : Test visuel Roborazzi validant la conformité graphique de l'en-tête de l'application.
-

